FitBuddy

AI Fitness Plan Generator using Gemini Models

Project Documentation

FastAPI • Google Gemini 1.5 Pro / Flash • SQLAlchemy • Jinja2

SmartBridge | SmartInternz

Table of Contents

1. Project Description
2. Application Scenarios
3. Technical Architecture
4. Prerequisites
5. Project Workflow Overview
6. Milestone 1 – Model Selection & Architecture
7. Milestone 2 – Core Functionality Development
8. Milestone 3 – Routing Layer (routes.py)
9. Milestone 4 – Frontend Development
10. Milestone 5 – Deployment
11. Application Walkthrough
12. Conclusion

1. Project Description

FitBuddy is a web-based application that uses AI to generate personalized workout plans and nutrition tips based on a user's fitness goals, such as weight loss, muscle gain, or general wellness. Built with FastAPI and integrated with Google's Gemini AI models, FitBuddy simplifies the process of creating structured and effective fitness routines.

The application takes in user information — such as age, weight, goal, and preferred workout intensity — and generates a customized 7-day fitness plan. Users can also submit feedback to update and refine their plan using AI. A complementary nutrition or recovery tip is also provided based on their selected goal.

With a clean user interface and database support via SQLite, FitBuddy ensures each user's data and generated plan are stored and accessible. It offers both API endpoints and an interactive HTML interface for ease of use.

2. Application Scenarios

Scenario 1 – Personalized Plan Generation

A user inputs personal details including name, age, weight, fitness goal (e.g., “weight loss” or “muscle gain”), and preferred workout intensity (“high”, “medium” or “low”) to receive a personalized 7-day workout plan. The system generates a day-by-day schedule tailored to the user's fitness objective and experience level, enhancing motivation and adherence through structured, goal-specific routines.

Scenario 2 – Feedback-Driven Plan Updates

A user submits feedback on their existing workout plan, such as “more focus on cardio” or “include more rest days.” The system uses this input to regenerate an updated plan, refining it based on the user's evolving needs and keeping them engaged with dynamic fitness support.

Scenario 3 – Nutrition & Recovery Tips

A user requests a nutrition or recovery tip based on their selected fitness goal. The platform delivers a concise, relevant suggestion (e.g., “Include protein in your post-workout meal” for muscle gain) that complements the workout plan and helps align diet and recovery with training.

Scenario 4 – Administrative Oversight

An admin or coach accesses the platform to view all registered users and their workout plans through a dedicated dashboard, tracking individual progress, comparing original and updated plans, and monitoring activity — streamlining oversight for trainers managing multiple clients or students.

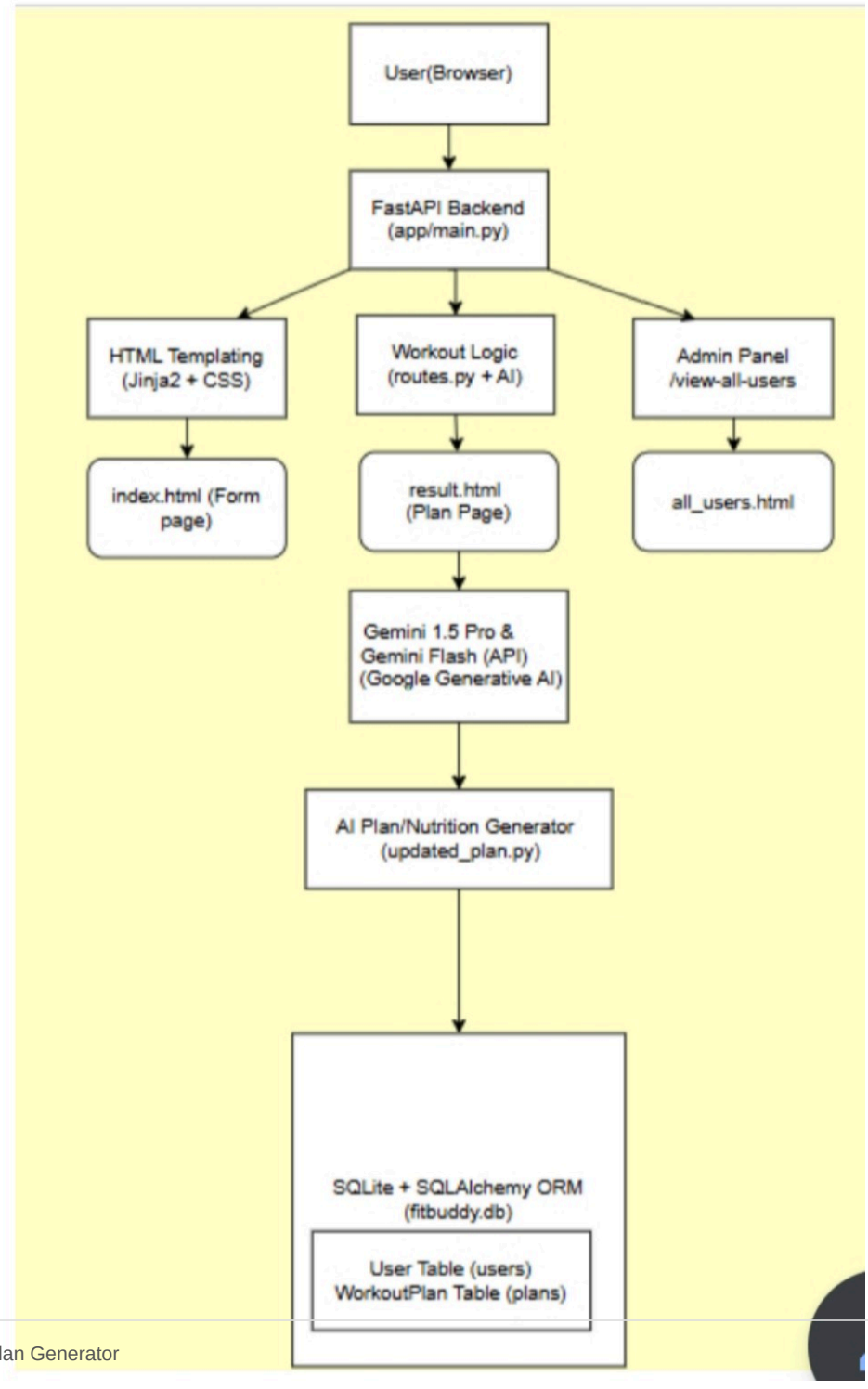
3. Technical Architecture

FitBuddy follows a modular FastAPI-based architecture:

- **Frontend:** HTML + Jinja2 templates for user input and display
- **Backend:** FastAPI for route handling and application logic
- **AI Layer:** Google Gemini APIs (Gemini 1.5 Pro & Gemini Flash)
- **Database:** SQLite with SQLAlchemy ORM for persistence

Request Flow

User (Browser) → FastAPI Backend (app/main.py) → three parallel layers: HTML Templating (Jinja2 + CSS via index.html), Workout Logic (routes.py + AI, producing result.html), and Admin Panel (/view-all-users →



all_users.html). The Workout Logic layer calls Gemini 1.5 Pro & Gemini Flash (Google Generative AI) through an AI Plan/Nutrition Generator (updated_plan.py), which persists results to SQLite + SQLAlchemy ORM (fitbuddy.db) across a **User** table (users) and a **WorkoutPlan** table (plans).

Project Folder Structure

```
fitbuddy/

|-- requirements.txt              # Project dependencies

|-- app/

    |-- main.py                   # FastAPI entry point

    |-- routes.py                 # Core route handlers

    |-- database.py               # SQLAlchemy models and DB logic

    |-- schemas.py                # Pydantic models for validation

    |-- gemini_generator.py       # Gemini Pro - workout plan generator

    |-- gemini_flash_generator.py # Gemini Flash - nutrition tips

    |-- updated_plan.py           # Feedback-based plan updater

    |-- nutrition.py              # Nutrition-specific logic (optional)

    |-- templates/

    |   |-- index.html            # User input form

    |   |-- result.html           # Workout plan, tip, feedback

    |   |-- all_users.html        # Admin dashboard

    |-- static/images/

    |   |-- gym-bg.jpg            # Gym-themed background

    |-- fitbuddy.db               # Local SQLite database (auto-generated)
```

Responsibilities

- **Frontend:** index.html (input form), result.html (plan, tip, feedback), all_users.html (admin panel)
- **Backend:** receives form data, calls Gemini models, stores user info and plans, renders templates
- **AI integration:** workout generation & feedback updates → Gemini Pro; nutrition tips → Gemini Flash

4. Prerequisites

- **FastAPI Framework Knowledge** – FastAPI Documentation
- **Gemini API Familiarity** – Google Generative AI Documentation
- **HTML, CSS & Jinja2 Templating** – W3Schools tutorials
- **Python Programming Proficiency** – Python Official Docs
- **Version Control with Git** – Git Documentation
- **SQLAlchemy & SQLite Basics** – SQLAlchemy Docs
- **Environment Setup** (pip, virtualenv, or conda) – Virtualenv Guide
- **Uvicorn ASGI Server** – Uvicorn Docs

5. Project Workflow Overview

Milestone	Focus
1. Model Selection & Architecture	Select the AI model; define app architecture; set up dev environment
2. Core Functionality Development	Build workout generation, nutrition tips, and feedback-based updates
3. routes.py Development	Implement FastAPI routing logic that ties the app together
4. Frontend Development	Design UI and wire dynamic Jinja2 templates
5. Deployment	Prepare, run, and verify local deployment

6. Milestone 1 – Model Selection & Architecture

This milestone focuses on selecting the appropriate generative AI models from Google's Gemini family for FitBuddy, evaluating performance and capabilities to ensure alignment with the application's core functionalities — generating personalized workout plans, updating them based on feedback, and providing tailored nutrition or recovery advice.

6.1 Activity 1.1 – Research & Select the Generative AI Model

FitBuddy's AI model needed to:

- Generate structured 7-day workout plans
- Provide concise and actionable nutrition tips
- Update workout plans based on user feedback
- Perform well in real-time, web-based interaction

Model Evaluation

Several Hugging Face models were initially evaluated:

- T5 & BART – for text generation (workout/nutrition)
- DistilBERT – for potential intent classification
- Bloom – for general-purpose generation

While these models showed promise for basic text generation, they required hosting on Hugging Face or a local GPU setup, plus fine-tuning or complex prompting to achieve structured plans.

Final Model Selection

After comparative testing, Google Gemini models were selected for production:

- **Gemini 1.5 Pro** – used for structured 7-day workout generation and plan updating; delivers rich, context-aware, formatted responses reliably
- **Gemini Flash** – used for generating fast, practical nutrition tips; lightweight and efficient for quick API calls

Reasons for choosing Gemini:

- API-based, easy to integrate with FastAPI
- Naturally understands structured formatting (days, sections)
- High accuracy with minimal prompt tuning
- Scalable and fast for feedback-based revisions

6.2 Activity 1.2 – Define the Application Architecture

FitBuddy follows a modular FastAPI-based architecture:

- **Frontend:** HTML + Jinja2 templates for user input and display
- **Backend:** FastAPI for route handling and logic
- **AI Layer:** Google Gemini APIs (Pro & Flash)
- **Database:** SQLite with SQLAlchemy for persistence

Frontend responsibilities – users interact via: index.html (input form), result.html (plan, tip, feedback), all_users.html (admin panel).

Backend responsibilities – receives form data; calls Gemini models; stores user info and plans; renders dynamic templates.

AI integration points – workout generation & feedback updates use Gemini Pro; nutrition tips use Gemini Flash.

6.3 Activity 1.3 – Set Up the Development Environment

```
# Create virtual environment

python -m venv fitbuddy-env

fitbuddy-env\Scripts\activate      # Windows


# Install required libraries

pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai


# Run the server

uvicorn app.main:app --reload

# Visit http://127.0.0.1:8000 for the app, /docs for API testing
```

Project folders were set up as: /app (Python files), /templates (HTML), /static/images (assets).

7. Milestone 2 – Core Functionality Development

7.1 Activity 2.1 – Develop the Core Functionalities

Workout Plan Generation

Function: generate_workout_gemini() | **File:** gemini_generator.py

Implements the core logic to generate a personalized 7-day workout plan using user inputs such as goal (e.g., muscle gain, weight loss) and intensity (low, medium, high). The prompt is sent to Gemini 1.5 Pro, which returns a structured, day-wise plan including a warm-up (5–10 mins), main workout (exercise details, sets, reps), and a cooldown or recovery tip. The output is returned to routes.py and rendered on the result.html page using <pre> blocks for formatting.

```
def generate_workout_gemini(user_input):

    prompt = f"""

    You are a professional fitness trainer.

    Create a personalized, structured 7-day workout plan for someone with

    the goal of **{user_input['goal']}**, and prefers **{user_input['intensity']}**

    intensity workouts.

    Each day must include: warm-up, main workout, cooldown/recovery tip.

    """

    try:

        response = model.generate_content(prompt)

        return response.text

    except Exception as e:

        return f"Error: {e}"
```

Nutrition Tip Generation

Function: generate_nutrition_tip_with_flash() | **File:** gemini_flash_generator.py

Generates a concise, practical nutrition or recovery tip tailored to the user's selected goal using Gemini Flash — fast, friendly advice such as hydration tips or protein suggestions, displayed beneath the workout plan in result.html.

```
def generate_nutrition_tip_with_flash(goal: str) -> str:
    """
    Generate a nutrition or recovery tip using Gemini Flash based on the user's fitness goal.

    Args:
        goal (str): User's fitness goal - "weight loss", "muscle gain", or "general fitness".

    Returns:
        str: Generated tip.
    """
    prompt = (
        f"Give one clear, helpful nutrition or recovery tip for someone focused on '{goal}'. "
        "The tip should be practical, friendly, and easy to understand."
    )

    try:
        response = model.generate_content(prompt)
        return response.text.strip()

    except Exception as e:
        return f"Error generating tip: {str(e)}"
```

Feedback-Based Plan Updating

Function: update_workout_plan() | **File:** updated_plan.py

Lets the user revise their original plan by submitting feedback (e.g., ·Add yoga·, ·Include more cardio·). The original plan and feedback are passed to Gemini 1.5 Pro, which returns a modified plan; the updated plan is stored and displayed alongside a new nutrition tip.

U

```
def update_workout_plan(original_plan: str, user_feedback: str) -> str:
    """
    Use Gemini 1.5 Pro to update the workout plan based on user feedback.
    """
    prompt = f"""
    You are a professional fitness trainer assistant.

    Here's the original 7-day workout plan:
    {original_plan}

    User Feedback:
    "{user_feedback}"

    Based on the feedback, revise the relevant parts of the workout plan. Keep the format and rest of the plan unchanged if not needed.
    """
    try:
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"Error updating plan: {e}"
```

```
def save_user(user_id: int, name: str, age: int, weight: float, goal: str, intensity: str):
    db = SessionLocal()
    existing = db.query(User).filter_by(id=user_id).first()
    if existing:
        # Update existing user info
        existing.name = name
        existing.age = age
        existing.weight = weight
        existing.goal = goal
        existing.intensity = intensity
    else:
        # Create a new user
        user = User(
            id=user_id,
            name=name,
            age=age,
            weight=weight,
            goal=goal,
            intensity=intensity,
            schedule=7  # default schedule or logic
        )
        db.add(user)
    db.commit()
    db.close()
```

```
#stores the plan in the database
def save_plan(user_id: int, plan: str):
    db = SessionLocal()
    workout = WorkoutPlan(user_id=user_id, original_plan=plan)
    db.add(workout)
    db.commit()
    db.close()
```

```
# Update the plan based on feedback
def update_plan(user_id: int, updated_text: str):
    db = SessionLocal()
    workout = db.query(WorkoutPlan).filter_by(user_id=user_id).first()
    if workout:
        workout.updated_plan = updated_text
        db.commit()
    db.close()
```

```
# Fetch the original plan for a user
def get_original_plan(user_id: int):
    db = SessionLocal()
    plan = db.query(WorkoutPlan).filter(WorkoutPlan.user_id == user_id).first()
    return plan.original_plan if plan else None
```

Functions: save_user(), save_plan(), update_plan(), get_original_plan(), get_user() | **File:** database.py

All user information (ID, name, age, weight, goal, intensity) and AI-generated plans are stored in a SQLite database via SQLAlchemy ORM. Feedback-based updates are saved as a separate field to preserve both the original and updated versions.

```
def save_plan(user_id: int, plan: str):

    db = SessionLocal()

    workout = WorkoutPlan(user_id=user_id, original_plan=plan)

    db.add(workout); db.commit(); db.close()


def update_plan(user_id: int, updated_text: str):

    db = SessionLocal()

    workout = db.query(WorkoutPlan).filter_by(user_id=user_id).first()

    if workout:

        workout.updated_plan = updated_text

        db.commit()

    db.close()
```

Admin View of All Users

Route: /view-all-users | **Template:** all_users.html

Displays a list of all users and their respective plans, rendered with Jinja2 using a {% for user in users %} loop, letting administrators see original and updated plans in a structured table format.

```python
# 8. Web: View all users & their plans
@router.get("/view-all-users", response_class=HTMLResponse)
def view_all_users(request: Request):
    db = SessionLocal()
    users = db.query(User).all()
    user_data = []
    for user in users:
        plan = db.query(WorkoutPlan).filter(WorkoutPlan.user_id == user.id).first()
        user_data.append({
            "id": user.id,
            "name": user.name,
            "age": user.age,
            "weight": user.weight,
            "goal": user.goal,
            "intensity": user.intensity,
            "original_plan": plan.original_plan if plan else "N/A",
            "updated_plan": plan.updated_plan if plan and plan.updated_plan else "Not updated"
        })
    db.close()
    return templates.TemplateResponse("all_users.html", {
        "request": request,
        "users": user_data
    })
```

7.2 Activity 2.2 · Implement the FastAPI Backend

Define routes in FastAPI: all routing logic is handled in routes.py; each route is linked to the corresponding function from Activity 2.1.

Process user input:

- HTML form created in index.html
- Users enter: username, user_id, age, weight, goal, and intensity
- FastAPI captures inputs using Form(...) parameters
- Pydantic schemas (UserInput, FeedbackRequest) validate structure

Integrate Gemini API calls · each route handler connects directly to its Gemini-powered function:

- /generate-workout • generate_workout_gemini() for the plan; generate_nutrition_tip_with_flash() for the tip
- /submit-feedback • retrieves the plan and sends original + feedback to update_workout_plan()

All AI-generated responses are returned cleanly and injected into the frontend templates for display.

8. Milestone 3 – Routing Layer (routes.py)

This milestone focuses on building the routing logic inside routes.py, the core operational layer of the FitBuddy application. It bridges the frontend templates, AI generation modules (Gemini Pro and Flash), and the SQLite database. Each route defines how user inputs are collected, processed, and responded to, creating a dynamic, personalized experience for fitness planning and feedback-based updates.

8.1 Activity 3.1 – Core Routes

- **/** — displays the user form via index.html
- **/generate-workout** — processes user input and generates a personalized workout plan
- **/submit-feedback** — updates the workout plan based on user feedback
- **/view-all-users** — displays an admin dashboard showing all users and their plans

Route Handlers

- **/ – Home Route:** returns the homepage (index.html); no form processing logic
- **/generate-workout – Plan Generator:** receives input via Form(...), builds a UserInput model, calls generate_workout_gemini() and generate_nutrition_tip_with_flash(), stores user details and plan with save_user()/save_plan(), and returns result.html with dynamic data
- **/submit-feedback – Update Plan with Feedback:** captures feedback and user_id, retrieves the original plan, calls update_workout_plan() (Gemini 1.5 Pro) to revise it, saves the revision with update_plan(), and renders the updated result on result.html
- **/view-all-users – Admin View:** fetches all user and plan records and passes them to all_users.html, where admins can view username, age, weight, goal, intensity, and both original and updated workout plans

```
# Generate workout using Gemini Pro

@router.post("/generate-workout/gemini")

async def generate_gemini_workout(request: WorkoutRequest):

    try:

        result = generate_workout_gemini({

            "goal": request.goal, "intensity": request.intensity

        })

        return {"model": "gemini-pro", "workout_plan": result}

    except Exception as e:

        raise HTTPException(status_code=500, detail=str(e))


# Generate nutrition tip using Gemini Flash

@router.get("/nutrition-tip")

def get_flash_tip(goal: str):

    tip = generate_nutrition_tip_with_flash(goal)
```

```
    return {"goal": goal, "nutrition_tip": tip}


# Update plan based on feedback

@router.post("/update-plan/{user_id}", response_model=dict)

def update_user_plan(user_id: int, data: FeedbackRequest):

    original = get_original_plan(user_id)

    if not original:

        return {"error": "Original plan not found for this user."}

    updated = update_workout_plan(original, data.feedback)

    update_plan(user_id, updated)

    return {"updated_plan": updated}
```

All responses are returned to the user through result.html, maintaining clarity using <pre> blocks for formatting.

Saving user info and plan generation route

```python
# 3. API: Save user info & generate plan
@router.post("/generate-plan")
def generate_plan(user_data: UserInput):
    try:
        save_user(
            user_id=user_data.user_id,
            name=user_data.username,
            age=user_data.age,
            weight=user_data.weight,
            goal=user_data.goal,
            intensity=user_data.intensity
        )
        plan = generate_workout_gemini({
            "goal": user_data.goal,
            "intensity": user_data.intensity
        })
        save_plan(user_data.user_id, plan)
        return {
            "message": "Workout plan generated and saved successfully!",
            "workout_plan": plan
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Something went wrong: {str(e)}")
```

Submit Feedback and update plan route

```python
# 4. API: Update workout plan based on user feedback
@router.post("/update-plan/{user_id}", response_model=dict)
def update_user_plan(user_id: int, data: FeedbackRequest):
    original = get_original_plan(user_id)
    if not original:
        return {"error": "Original plan not found for this user."}
    updated = update_workout_plan(original, data.feedback)
    update_plan(user_id, updated)
    return {"updated_plan": updated}
```

9. Milestone 4 – Frontend Development

This milestone focuses on developing a user-friendly, visually structured interface for FitBuddy using HTML, CSS, and Jinja2 templating. Each template is dynamically rendered through FastAPI, ensuring users can easily submit input, receive personalized plans, provide feedback, and allow administrators to manage data effectively.

9.1 Activity 4.1 – Designing and Developing the User Interface

Base HTML Structure

- index.html is the main entry point for user input, capturing username, user_id, age, weight, goal, and intensity
- Each field is properly labeled and grouped using semantic HTML
- A gym-themed background image, Google Fonts (Roboto), and bold typography create a clean, fitness-oriented interface
- Navigation is kept minimal for focus, with a clear user flow from input to output

Responsive Layout with CSS

- Embedded CSS styles applied directly in each HTML file
- Flexbox layouts center content vertically and horizontally
- Media queries ensure mobile responsiveness
- Consistent button styling with hover effects
- Input fields and result blocks use padding, shadows, and rounded corners for a modern look

Three Core Pages

- **index.html** – user input form for fitness goal, intensity, and personal information
- **result.html** – displays the 7-day AI-generated workout plan, a nutrition tip, and a feedback form
- **all_users.html** – admin panel to view users and review both original and updated plans

Templates are modular, UI transitions smoothly between steps, and forms submit to specific FastAPI endpoints via POST.

9.2 Activity 4.2 – Creating Dynamic Templates with Jinja2

```
from fastapi.templating import Jinja2Templates

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

TEMPLATE_DIR = os.path.join(BASE_DIR, "templates")

templates = Jinja2Templates(directory=TEMPLATE_DIR)


return templates.TemplateResponse("result.html", {

    "request": request, "username": username, "user_id": user_id,

    "age": age, "weight": weight, "goal": goal, "intensity": intensity,

    "workout_plan": plan, "nutrition_tip": nutrition_tip

})
```

Each route uses TemplateResponse to render the correct HTML with the passed context, allowing dynamic plan and nutrition-tip display after form submission.

- **index.html:** form elements use name="..." to bind to backend Form(...) parameters; submits to /generate-workout
- **result.html:** uses {{ username }}, {{ goal }}, {{ intensity }}, {{ workout_plan }}, {{ nutrition_tip }} to render user data and AI results; includes a feedback form posted to /submit-feedback
- **all_users.html:** loops over all user records with {% for user in users %} to render a table of original and updated plans

- Loops over all user records using:

```
</tr>
{% for user in users %}
<tr>
    <td>{{ user.id }}</td>
    <td>{{ user.name }}</td>
    <td>{{ user.age }}</td>
    <td>{{ user.weight }}</td>
    <td>{{ user.goal }}</td>
    <td>{{ user.intensity }}</td>
    <td><pre>{{ user.original_plan }}</pre></td>
    <td><pre>{{ user.updated_plan }}</pre></td>
</tr>
{% endfor %}
```

10. Milestone 5 – Deployment

The focus of this milestone is deploying FitBuddy on a local system using FastAPI and Uvicorn. The app uses HTML templates rendered with Jinja2, alongside AI services powered by Gemini. This milestone ensures all environment dependencies are correctly installed and the app runs smoothly on a local server, preparing it for future deployment on cloud platforms.

10.1 Activity 5.1 – Preparing the Application for Local Deployment

```
# Set up a virtual environment

python -m venv venv

source venv/bin/activate      # Windows: venv\Scripts\activate

pip install -r requirements.txt


# Configure environment variables (e.g. in a .env file, via python-dotenv)

GOOGLE_API_KEY=your_gemini_api_key_here
```

All user interactions are facilitated via Jinja2-rendered templates: index.html collects user inputs, result.html displays the generated AI plan, and all_users.html lists all user records for admin review.

10.2 Activity 5.2 – Testing and Verifying Local Deployment

The FastAPI development server is started with `uvicorn main:app --reload`. Once running, the application is accessible at `http://127.0.0.1:8000`, which loads the main user interface for entering details and generating personalized fitness plans. The interactive API documentation provided by FastAPI is available at `http://127.0.0.1:8000/docs`, allowing developers to explore and test available endpoints.

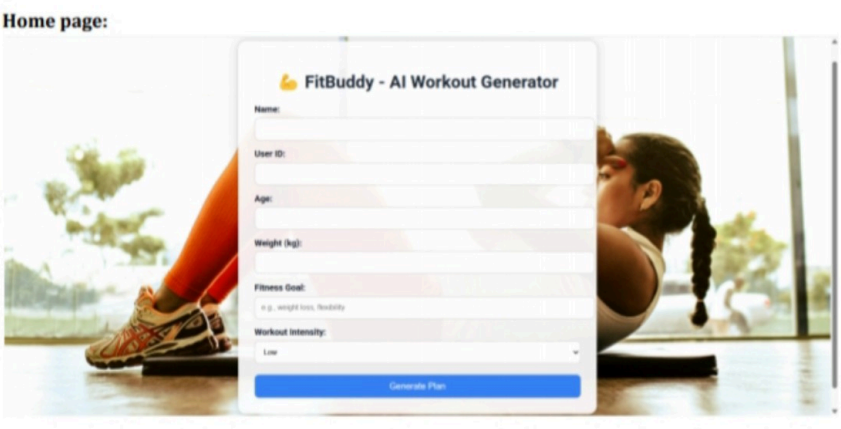
11. Application Walkthrough

11.1 Home Page

The homepage of FitBuddy features a clean, engaging interface for a seamless experience. A centered form allows users to enter their Name, User ID, Age, Weight (kg), Fitness Goal (e.g., weight loss, flexibility), and Workout Intensity (Low, Medium, High). Submitting the form triggers the AI-powered backend to generate a customized workout and nutrition plan tailored to the user's profile. The fitness-themed background reinforces the app's purpose and motivates users to take charge of their health journey, while the intuitive design lets users of all experience levels get started quickly.

Exploring Website's web pages:

Home page:



Description: The homepage of **FitBuddy – AI Workout Generator** features a clean and engaging user interface designed to provide a seamless experience for users seeking personalized fitness plans. The form prominently displayed in the center allows users to enter essential information including:

- **Name**
- **User ID**
- **Age**
- **Weight (kg)**
- **Fitness Goal** (e.g., weight loss, flexibility)
- **Workout Intensity** (Low, Medium, High)

11.2 Personalized Workout Plan Page

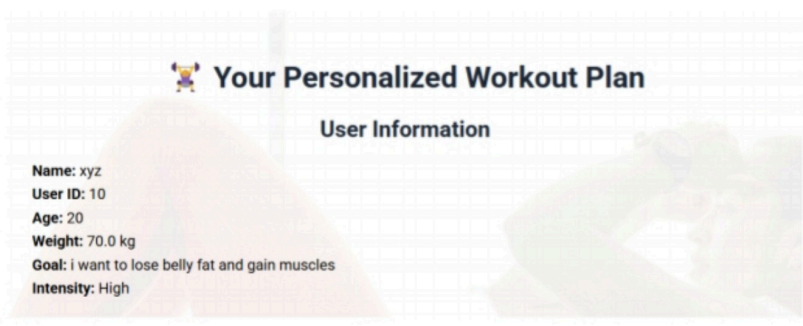
After submission, the app navigates to a personalized workout plan page. A summary of the user's input — Name, User ID, Age, Weight, Fitness Goal, and Selected Intensity — is shown at the top, so users feel acknowledged and the AI's recommendations feel clearly tailored to their fitness journey.

Once users submit this data, they can click the **"Generate Plan"** button, which triggers the AI-powered backend to generate a customized workout and nutrition plan tailored to their goals and fitness profile. The visual appeal of the background image featuring an active individual reinforces the fitness-centric theme of the application, motivating users to take charge of their health journey.

This intuitive design ensures users of all experience levels can quickly get started with minimal effort.

Personalized workout generation page:

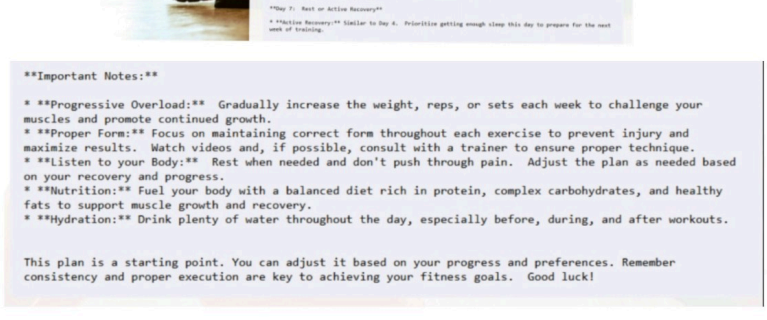
User Information:



Workout Plan: FitBuddy generates a personalized, day-wise workout plan tailored to the user's goal, experience level, and preferred intensity. Each day is outlined with a clear focus (full body, upper/lower body, cardio, core, or flexibility), and every day lists exercises with sets, repetitions/duration, and rest intervals, plus warm-up and cool-down guidance — delivering a goal-oriented, easy-to-follow schedule that promotes consistency and progress.

Workout Plan:





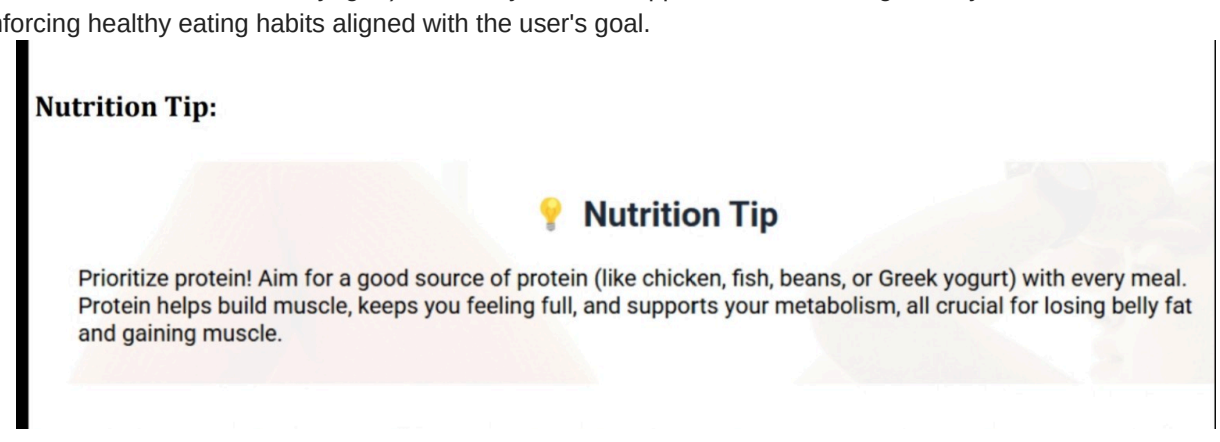
```
**Important Notes:**

* **Progressive Overload:**  Gradually increase the weight, reps, or sets each week to challenge your
muscles and promote continued growth.
* **Proper Form:** Focus on maintaining correct form throughout each exercise to prevent injury and
maximize results.  Watch videos and, if possible, consult with a trainer to ensure proper technique.
* **Listen to your Body:**  Rest when needed and don't push through pain.  Adjust the plan as needed based
on your recovery and progress.
* **Nutrition:** Fuel your body with a balanced diet rich in protein, complex carbohydrates, and healthy
fats to support muscle growth and recovery.
* **Hydration:** Drink plenty of water throughout the day, especially before, during, and after workouts.


This plan is a starting point. You can adjust it based on your progress and preferences. Remember
consistency and proper execution are key to achieving your fitness goals.  Good luck!
```

Nutrition Tip: A concise, practical dietary tip complements the workout plan, e.g. prioritizing a protein source (chicken, fish, beans, or Greek yogurt) with every meal to support muscle building, satiety, and metabolism — reinforcing healthy eating habits aligned with the user's goal.

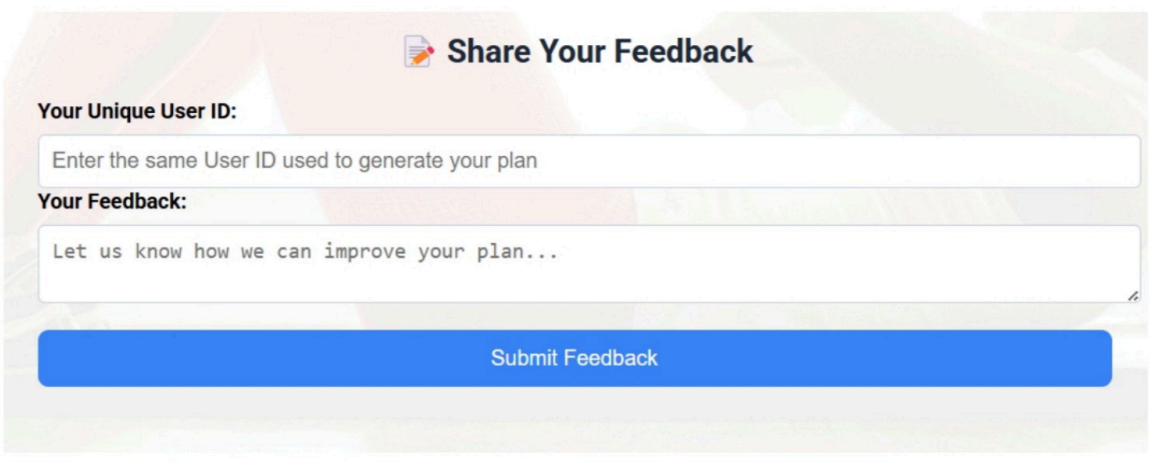
Nutrition Tip:



11.3 Feedback Page

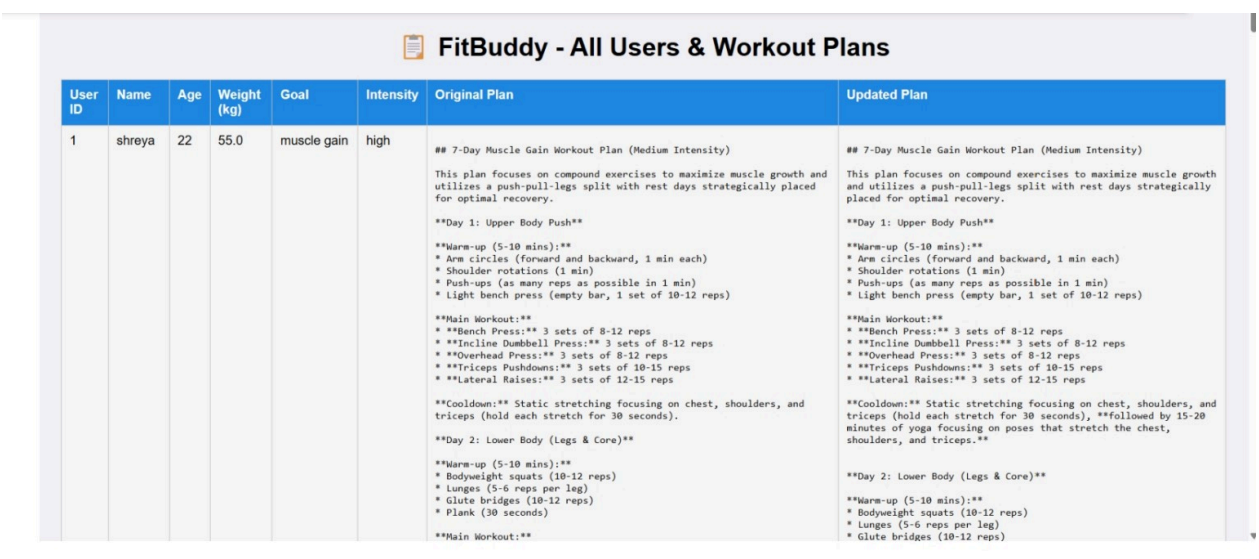
The Feedback Page lets users personalize and refine their fitness journey by submitting feedback on their current plan. Users enter their unique User ID and specific feedback about effectiveness, difficulty, or preferences. The system processes this feedback to intelligently update the workout plan, and a confirmation message ("Your plan has been updated based on your feedback!") confirms the change — a dynamic feedback loop that keeps each plan adaptive and tailored to the user's evolving needs.

Feedback Page:



11.4 View All Users (Admin) Page

The View All Users page gives an overview of the entire user database: each registered user's personal details (name, age, fitness goals) alongside their original workout plan and any updated plan generated from feedback. This provides a transparent view of how each user's plan has evolved over time, helping administrators or developers monitor user progress and engagement effectively.



12. Conclusion

FitBuddy – AI Fitness Plan Generator is a comprehensive, user-centric web application that showcases the powerful synergy between artificial intelligence and health-tech solutions. Designed to assist individuals in achieving their fitness goals, FitBuddy uses Google's Gemini 1.5 Pro and Gemini Flash models to generate personalized 7-day workout plans and daily nutrition or recovery tips based on user inputs such as age, weight, fitness goal, and preferred workout intensity.

The application is built using the FastAPI framework for backend logic and API handling, SQLAlchemy with SQLite for persistent data storage, and Jinja2 templating for a clean, responsive frontend. Upon submitting their details on the homepage, users receive a tailored workout plan that includes warm-ups, main workouts, cooldowns, and goal-aligned tips. What sets FitBuddy apart is its dynamic feedback feature: users can provide input on the effectiveness or difficulty of their current plan, and the system responds by intelligently updating the plan using generative AI, ensuring ongoing customization and engagement.

The application also includes an administrative view where all registered users and their respective plans (original and updated) are displayed in a structured format, supporting transparency, monitoring, and further optimization — making FitBuddy suitable for both individual users and coaches or institutions managing multiple fitness profiles.

Overall, FitBuddy stands as a scalable and adaptable AI-powered solution that not only personalizes the fitness journey but also evolves with the user's progress and preferences, demonstrating how cutting-edge technology can be integrated into everyday wellness routines to promote healthy, consistent, and goal-oriented lifestyles.